

```
akshi@akshita:~$ mkdir process_lab
cd process_lab
mkdir: process_lab: File exists
```

```
akshi@akshita:~/process_lab$ nano multi_child_wait.c
akshi@akshita:~/process_lab$ ls
multi_child_wait.c      process_states.c      zombie_demo            zombie_fixed.c
multi_child_waitpid.c  wait_demo              zombie_demo.c
process_states          waitpid_demo           zombie_fixed
```

```
akshi@akshita:~/process_lab$ ./waitpid_demo
Parent PID = 2037
Child 0: PID=2038, PPID=2037, starting work...
Child 1: PID=2039, PPID=2037, starting work...
Child 2: PID=2040, PPID=2037, starting work...
Child 2: PID=2040 finished work, exiting.
Child 1: PID=2039 finished work, exiting.
Child 0: PID=2038 finished work, exiting.
Parent: reaped child #0 (PID=2038) in the order I asked, exit code=100
Parent: reaped child #1 (PID=2039) in the order I asked, exit code=101
Parent: reaped child #2 (PID=2040) in the order I asked, exit code=102
Parent: all children finished. Exiting.
```

```
akshi@akshita:~/process_lab$ ./wait_demo
Parent PID = 2050
Child 0: PID=2051, PPID=2050, starting work...
Child 1: PID=2052, PPID=2050, starting work...
Child 2: PID=2053, PPID=2050, starting work...
Child 0: PID=2051 finished work, exiting.
Parent: reaped child PID=2051, exit code=100
Child 1: PID=2052 finished work, exiting.
Parent: reaped child PID=2052, exit code=101
Child 2: PID=2053 finished work, exiting.
Parent: reaped child PID=2053, exit code=102
Parent: all children finished. Exiting.
```

```
akshi@akshita:~/process_lab$ ./zombie_fixed
Parent: PID=2063, child PID=2064 – now properly waiting for it.
Child: PID=2064, exiting immediately.
Parent: reaped child PID=2064, exit code=0. No zombie created.
```

```
akshi@akshita:~$ ps -el | grep Z
F S  UID      PID     PPID  C PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
akshi@akshita:~$
```

nano multi_child_wait.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

#define NUM_CHILDREN 3

int main() {
    pid_t pid;

    printf("Parent PID = %d\n", getpid());

    for (int i = 0; i < NUM_CHILDREN; i++) {
        pid = fork();

        if (pid < 0) {
            perror("fork failed");
            exit(1);
        }

        if (pid == 0) {
            printf("Child %d: PID=%d, PPID=%d, starting work...\n",
                i, getpid(), getppid());
            sleep(i + 1);
            printf("Child %d: PID=%d finished work, exiting.\n", i, getpid());
            exit(100 + i);
        }
    }

    int status;
    pid_t finished_pid;
    for (int i = 0; i < NUM_CHILDREN; i++) {
        finished_pid = wait(&status);
```

```

        if (WIFEXITED(status)) {
            printf("Parent: reaped child PID=%d, exit code=%d\n",
                finished_pid, WEXITSTATUS(status));
        }
    }

    printf("Parent: all children finished. Exiting.\n");
    return 0;
}

```

nano multi_child_waitpid.c

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

#define NUM_CHILDREN 3

int main() {
    pid_t pids[NUM_CHILDREN];

    printf("Parent PID = %d\n", getpid());

    for (int i = 0; i < NUM_CHILDREN; i++) {
        pid_t pid = fork();

        if (pid < 0) {
            perror("fork failed");
            exit(1);
        }

        if (pid == 0) {
            printf("Child %d: PID=%d, PPID=%d, starting work...\n",
                i, getpid(), getppid());
            sleep(NUM_CHILDREN - i);
            printf("Child %d: PID=%d finished work, exiting.\n", i, getpid());
            exit(100 + i);
        } else {
            pids[i] = pid;
        }
    }
}

```

```

int status;
for (int i = 0; i < NUM_CHILDREN; i++) {
    pid_t result = waitpid(pids[i], &status, 0);
    if (WIFEXITED(status)) {
        printf("Parent: reaped child #%d (PID=%d) in the order I asked, exit code=%d\n",
            i, result, WEXITSTATUS(status));
    }
}

printf("Parent: all children finished. Exiting.\n");
return 0;
}
nano zombie_demo.c

```

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        exit(1);
    }

    if (pid == 0) {
        printf("Child: PID=%d, exiting immediately.\n", getpid());
        exit(0);
    } else {
        printf("Parent: PID=%d, child PID=%d has exited but I will NOT wait().\n",
            getpid(), pid);
        printf("Parent: sleeping 30 seconds. Quick! In another terminal run:\n");
        printf("    ps -el | grep Z\n");
        printf("or: ps -p %d -o pid,ppid,stat,cmd\n", pid);

        sleep(30);

        printf("Parent: done sleeping, exiting now (zombie will be cleaned up).\n");
    }

    return 0;
}

```

nano zombie_fixed.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        exit(1);
    }

    if (pid == 0) {
        printf("Child: PID=%d, exiting immediately.\n", getpid());
        exit(0);
    } else {
        printf("Parent: PID=%d, child PID=%d — now properly waiting for it.\n",
            getpid(), pid);

        int status;
        waitpid(pid, &status, 0);

        if (WIFEXITED(status)) {
            printf("Parent: reaped child PID=%d, exit code=%d. No zombie created.\n",
                pid, WEXITSTATUS(status));
        }

        sleep(30);
        printf("Parent: exiting now.\n");
    }

    return 0;
}
```

```
gcc multi_child_wait.c -o wait_demo
gcc multi_child_waitpid.c -o waitpid_demo
gcc zombie_demo.c -o zombie_demo
gcc zombie_fixed.c -o zombie_fixed
```

./wait_demo
./waitpid_demo
./zombie_demo
./zombie_fixed